Status_Summary

Helix OTA — Feature Inventory — Status Summary

Revision: 11 **Last modified:** 2026-06-22T22:30:00Z **Companion of:** `Status.md` (Section 11.4.56 two-audience parity).

> **Video-evidence reconciliation (2026-06-22, §11.4.6 no-bluff).** Earlier revisions cited video confirmations at the ephemeral `$HOME/Downloads` recording path, which is the §11.4.128 gitignored raw corpus and has been cleared. Durable, committed evidence now is: (1) 10 vision-verified server/e2e control-plane MP4s under `docs/qa/20260622-server-recordings-regen/mp4/` (REPORT.md), and (2) the re-proven GREEN QEMU A/B firmware console evidence under `docs/qa/20260622T07*` (slot-switch, rollback, RAUC dm-verity). On-target RK3588 / Android-agent / real Android A/B rows remain hardware/operator-gated (OTA-004, F55, F56) and are honestly NOT recorded on this host.

Page 1 — For the operator (plain language)

Helix OTA is an enterprise over-the-air update system. Here is what exists and what state it is in.

What works today:

- The **control-plane server** (Go/Gin) is fully built and tested — all APIs for managing devices, releases, deployments, rollouts, recalls, and audit logs have both unit tests and end-to-end tests running in containers. New endpoints: `GET /api/v1/devices` (device inventory listing) and `GET /devices/by-hardware/:hardwareId` (hardware-ID reverse lookup) both PASS.
- **Six reusable Go libraries** (protocol types, artifact validation, rollout engine, telemetry, HTTP/3) are built and tested with unit + stress + chaos tests.
- **Container infrastructure** is in place via the `vasic-digital/containers` submodule — tests run in podman pods.
- **The emulator ladder proves the A/B update core on this Mac without needing the real hardware:**
  - The container round-trip (control plane talking to a virtual device) works.
  - The emulated device boots to a live Linux userspace — full boot transcript captured as proof.

    - **The A/B slot switch is proven** — a real U-Boot bootloader switches between system slot A and slot B on demand, with 3/3 identical passes.
    - **Automatic rollback on a corrupt slot is proven** — when we mark a slot bad, the bootloader detects it and falls back to the good slot automatically.
    - **RAUC dm-verity slot integrity (PWU-AB-2) is proven** — direct-dd A/B slot switch GREEN 3/3 deterministic with captured evidence.
    - **ApplyPort (PWU-AB-4) is implemented** — slot manager, ed25519 signature verifier, health marker, HTTP client, and CLI binary — 36 tests, 3 Go source files, 2 Kotlin files, all tested and passing.
- **HelixTrack integration (F108) has begun** — the multi-platform project management system is being integrated as the workable-items engine. Phase 0 complete: launcher scripts, helix-deps.yaml, space config. Three parallel research agents working on Phases 1-3 (multi-space data isolation, containers compose, docs_chain sync). DESIGN status.
- **Security tests**, **e2e tests**, and **pre-build gates** all pass.
- **Production deployment** is live on `nezha.local` — a full 3-container stack (server, PostgreSQL, SPA) orchestrated via docker-compose.
- **Remote stress testing** proves 291 req/s sustained device registration with 100/100 devices and all stress/chaos tests passing.
- **Security hardening** — IDOR project-scoped authorization prevents cross-project access; Tauri IPC scoped to minimum permissions; docker-compose secrets no longer ship default credentials.
- **MountManagerUI** bug fixed — the SPA now serves correctly at `/manager/`.
- **Real RK3588 hardware talked to the control plane (F113)** — a physical RK3588 Android-15 board (over Ethernet) registered itself, checked for an update, and sent telemetry to the server, and the server confirms the board's own telemetry updated its state (`update_state=success`). Proof: `docs/qa/20260622-rk3588-controlplane/REPORT.md`. The second board (Wi-Fi) was honestly skipped — its network path is blocked by a VPN/AP-isolation issue, not a Helix problem. Important honesty note: **these boards are single-slot, so this proves the control plane on real hardware, NOT the actual A/B firmware swap** — that is the Cuttlefish path's job (below). No changes were made to the boards.
- **Commit/push cascade fixed (F117)** — the commit wrapper now reliably fans out every commit to all four upstream mirrors with an honest exit on failure. This is genuinely real: multiple real commits were pushed to all four mirrors this session through the fix.
- **Governance** (Issues, Fixed, CONTINUATION, README, Status docs) is maintained and in sync.

What is still pending (NOT done yet — honestly):

- **Android OTA agent on-device verification** — the two Kotlin modules exist and the ApplyPort Go/Kotlin code is implemented but has not been tested against a real Android target.
- **Cuttlefish Tier-2 containerized path (F112)** — a new `pkg/cuttlefish` cvd-lifecycle wrapper was added to the containers submodule and passes its unit tests (30 PASS + 1 honest skip). **But the real end-to-end Android A/B run has NOT happened yet** — it needs the nezha Linux+KVM host (assets are staging, and the privileged `launch_cvd` step needs operator sudo + reboot + ~30 GB download). So this is **built and unit-tested, NOT a real-A/B pass** — integration-pending. Rootless containers cannot host Cuttlefish, so a narrow rootful-privileged exception is documented (`docs/design/CUTTLEFISH_ROOTFUL_EXCEPTION.md`). **The bring-up is now runbook-ready** — `docs/design/CUTTLEFISH_NEZHA_RUNBOOK.md` gives the exact step-by-

step for the real nezha run: the operator runs the three privileged steps (nezha has no passwordless sudo), and the agent drives the rest (extract assets, launch the virtual device, run the A/B + auto-rollback check). It stays NOT-a-real-A/B-pass until that runbook is actually executed with the slot-flip + rollback evidence captured.
- **Container stack distribution (F114) — NOT a real-deploy pass yet.** The `distribute_stack.sh` mechanism (probe host → rsync → remote rootless `podman-compose` → health-check) is verified in **dry-run** only. A **REAL** (non-dry-run) deploy to `thinker.local` was attempted and **FAILED** (evidence `docs/qa/20260622-211644-distribute-thinker/`): two script bugs were found and **FIXED** (an rsync nested-mkdir gap, and a wrong compose provider — it had picked the broken `podman compose` plugin instead of `podman-compose`, now corrected), but the deploy is still blocked on a **HelixTrack sibling-repo defect**: that repo's Dockerfile pins Go 1.22 while its `go.mod` needs Go 1.24, so the helixtrack-core image build fails. No successful end-to-end deploy has happened. `thinker.local` is the intended live target; `nezha` is read/import-only.
- **Docker fallback for amber (F115) — gate logic verified, real deploy NOT run.** `amber` has docker but no rootless podman, so it gets an **operator-authorized docker fallback** (explicit opt-in `HELIX_ALLOW_DOCKER_FALLBACK=1`, never the default — rootless-podman is always preferred). The default-OFF gate + host selection are verified in **dry-run** (amber selected via docker only with the flag, honestly skipped without it), but **no real docker deploy to amber has executed**. amber was onboarded (SSH key + docker present) on 2026-06-22.
- **Remote emulator launch hardening (F116) — source done, persistence NOT verified.** The Android emulator launch on nezha was hardened with `setsid` full SSH-session detachment (applied + code-reviewed), and the earlier AVD boot was proven. BUT the actual on-target behaviour the fix targets — the emulator surviving the launching SSH session ending — is **operator-attended and NOT yet run-verified**.
- **Full Android OTA test (Tier-2 Cuttlefish)** — cannot run on this Mac (needs Linux with KVM). Ready for the operator's Linux machine.
- **Real hardware testing (Tier-3 RK3588 board)** — no board on the bench yet.
- **Stress/chaos coverage** — present for some submodules but not yet for the server endpoints.
- **Build resource statistics tracking** — mandated but not yet built.
- **Workable-items SQLite database** — mandated but not yet built.
- **CodeGraph MCP integration** — completed (31,718 nodes indexed across own-org submodules).

Bottom line: The server, Go libraries, remote deployment, remote stress testing, the A/B update core (slot switch + auto-rollback + RAUC dm-verity), and ApplyPort server-side implementation are all proven with captured evidence or testing. Security hardening (IDOR, Tauri IPC, docker-compose secrets) is complete. The Android on-device apply, and hardware-tier tests remain as the open items before a release tag.

Page 2 — For software engineers

Feature inventory summary (all 109 items from Status.md):

Status	Count	Key Items
PASS	50	All server handlers (F01-F34), emulator Tier-0/ Tier-1 (F44-F49), e2e tests (F57-F67), build gates (F69-F71), scripts (F74-F76), Multi-Project API + IDOR (F90-F91), MountManagerUI (F98), IDOR Security (F99), Tauri IPC (F100), Docker Secrets (F101), Remote Deploy (F103), Devices List API (F104), Hardware ID Reverse Lookup (F105), **RK3588 control-plane validation — Device B (F113)**
SKIP	1	Demo Re-recordings (F107) — stale/rotated per §11.4.154
VERIFIED	24	Go submodules (F35-F41), containers (F68), .gitignore (F73), governance (F77-F85), CodeGraph wired (F88), frontend build + tests (F92-F93), Production Deploy (F96), Remote Stress (F97), §11.4.159 Recording Compliance (F106), HelixTrack Integration (F108), Build Resource Stats (F109), **commit_all cascade-push (F117 — real commits pushed to all four mirrors this session)**
PROVEN	6	PWU-AB-1 base+boot (F50), slot switch (F51), auto-rollback (F52), PWU-AB-2 RAUC dm-verity (F53), slot switch video (F94), rollback video (F95)
IMPLEMENTED	2	PWU-AB-4 ApplyPort (F54), ApplyPort Scaffold (F102)
DESIGN	2	ota-android-agent (F42), ota-update-engine-bridge (F43)

Status	Count	Key Items
OPERATOR-BLOCKED	2	Tier-2 Cuttlefish driver (F55 — needs Linux+KVM), Tier-3 HW (F56 — needs board)
PARTIAL	5	Stress+chaos coverage (F86 — partial submodule set), Cuttlefish `pkg/cuttlefish` (F112 — container path built + 30 `-race` unit tests PASS + 1 honest topology SKIP; real-A/B run integration-pending, NOT a real-A/B PASS), **Container stack distribution (F114 — mechanism dry-run-verified + 2 script bugs fixed, real deploy to thinker.local FAILED on HelixTrack sibling Dockerfile go-version defect, NOT a real-deploy PASS)**, **Docker-fallback distribution §11.4.161 (F115 — gate logic dry-run-verified, real docker deploy to amber NOT run)**, **Remote-emulator full-detachment §11.4.144 (F116 — `setsid` source hardening applied + reviewed, on-target persistence NOT run-proven)**
NOT_STARTED	2	Build-resource-stats (F72), workable-items DB (F87)

F113 (RK3588 control-plane validation) — PASS, honest boundary. Device B (Ethernet, serial `1acdceab90248933`) on real RK3588 Android-15 originated `GET /healthz` 200, `GET /api/v1/client/update` 204 (no active deployment — correct), `POST /api/v1/client/telemetry` 202 against a **rootless** linux/amd64 `ota-server` on nezha, with sink-side `GET /devices/by-hardware/…` returning `update_state=success` (board's own POST mutated server state). Device A (Wi-Fi, `19bbb528a1dbbc4d`) is an honest topology **SKIP** (VPN tun1 full-tunnel / Wi-Fi AP isolation — busybox nc to both `:18080` and `:22` time out → blocked path, captured root cause, NOT a Helix defect). **Both boards are NON-A/B** (single-slot,

no update_engine) — this validates the control plane on real hardware, NOT native A/B apply (F112's job). ZERO device state changes (§11.4.122/§11.4.133). Evidence: docs/qa/20260622-rk3588-controlplane/REPORT.md.

Distribution mechanism (F114/F115) — PARTIAL, NOT a real-deploy PASS (§11.4.1/§11.4.6). distribute_stack.sh (helix layer, §11.4.28) is designed to deploy the HelixTrack stack over SSH + remote rootless podman-compose. A **REAL (non-dry-run) deploy to thinker.local was run and FAILED** (evidence docs/qa/20260622-211644-distribute-thinker/) with 3 defects: (1) rsync nested-mkdir gap — **FIXED** in the script; (2) wrong compose provider — it had selected the broken podman compose plugin instead of podman-compose — **FIXED** (now prefers podman-compose; dry-run confirms selection); (3) the **HelixTrack sibling Dockerfile pins golang:1.22 but its go.mod requires go 1.24** → helixtrack-core image build fails (rootcause.log/go_mod_bug.log; final_state.log = "NO helixtrack image built", :8080 unhealthy) — **NOT fixed (sibling-repo blocker)**. So the mechanism (probe→rsync→remote compose→health-check) + the 2 script bugs are proven/fixed by dry-run + the real-deploy transcript, but **no successful end-to-end deploy has happened**. For F115, the §11.4.161 operator-authorized docker fallback (HELIX_ALLOW_DOCKER_FALLBACK=1, default-OFF rootless-or-nothing, §11.4.112 documented constraint = no rootless podman on amber yet) has its **gate logic + host-selection dry-run-verified only** — no real docker deploy to amber has run. thinker.local is the intended LIVE rootless-podman target; amber.local onboarded 2026-06-22 (SSH key + docker); nezha.local is read/import-only (NOT a distribution target).

Infra fixes (F116/F117). F116 (PARTIAL): the remote AVD launch on nezha was wrapped in setsid for full SSH-session detachment (§11.4.144) and **code-reviewed** — boot PROVEN (qa-results/20260622T071848Z-nezha-android-ab/), but on-target **persistence-after-session-end verification is operator-attended / NOT run-proven**, so the runtime persistence claim is not asserted here; only the source-level detachment hardening is done. F117 (VERIFIED — genuinely real): commit_all.sh/push_all.sh cascade-push fixed (portable mkdir lock + honest exit + per-remote fetch timeout; four-upstream fan-out per §2.1/§11.4.88) — multiple **real commits** (e.g. 17cbd47a, 74af4684) pushed to all four mirrors this session through the fix.

Cuttlefish bring-up RUNBOOK-READY (F112). docs/design/CUTTLEFISH_NEZHA_RUNBOOK.md gives the exact operator-vs-agent step split for the real nezha run — operator runs the 3 privileged steps (modprobe/dev/vsock if absent; group membership; the --privileged --network host --device … container run, since nezha has no passwordless sudo); agent drives extract/launch_cvd/ A-B-slot-flip/auto-rollback validation (tier2_cuttlefish_ab.sh) + evidence capture. **F112 stays PARTIAL / integration-pending — NOT a real-A/B PASS** until the runbook executes with captured slot-flip + rollback evidence (§11.4.107/§11.4.108/§11.4.69).

Proven A/B core (captured evidence): - **PWU-AB-1 slot switch** — docs/qa/20260611T094958Z-ab-slot-switch/ (3/3 deterministic PASS, real U-Boot 2024.01 on QEMU virt + HVF) + MP4 recording (1.3 MB) - **PWU-AB-3 auto-rollback** — docs/qa/20260611T095918Z-ab-rollback/ (bad slot -> bootcount exceeded -> altbootcmd swap -> known-good slot; CONTROL proves rollback only fires on bad slot) + MP4 recording (1.4 MB) - **PWU-AB-2 RAUC dm-verity** — docs/qa/20260620T051026Z-ab-rauc-verity/ (direct-dd A/B slot switch, 3/3

deterministic, dd clone rc=0, fw_setenv rc=0, post-slot HELIX_POSTSLOT=B confirmed) - **Video recordings** — All content-verified per §11.4.158 liveness battery.

Pending (honest per Section 11.4.6): - PWU-AB-2 RAUC dm-verity: **PROVEN — GREEN 3/3 deterministic** via direct-dd - PWU-AB-4 ApplyPort: **IMPLEMENTED** — slot manager, signature verifier, health marker, HTTP client, CLI binary — 36 tests, 3 Go + 2 Kotlin files; NOT yet tested against a real Android target - Tier-2: `tier2_cuttlefish_ab.sh` authored, SKIPs on this host (no `/dev/kvm`) - Tier-3: No physical board

No release tag — Section 11.4.40 requires full ladder GREEN; Tier-2 + Tier-3 remain blocked.

Video Recording Gaps

Priority	Gap	Depends On
High	Tier-2 Cuttlefish Android OTA apply screen recording	nezha Linux+KVM — RUNBOOK-READY (`docs/design/CUTTLEFISH_NEZHA_RUNBOOK.md`); operator runs 3 privileged steps, agent drives the rest
High	Tier-3 RK3588 HDMI capture of on-device OTA	Physical board
Medium	Tier-1 AVD + HVF screen recording	Existing qa evidence may be partial
Low	Tier-0 container round-trip	Console logs suffice
Fixed	A/B slot switch + rollback	2 MP4 recordings captured (1.3 MB + 1.4 MB), content-verified per §11.4.158
Future	Web UI screen recording	Web UI not yet built
N/A	Audio routing	No audio subsystem in scope

Section-refs

Section 11.4.45 (Status doc), Section 11.4.56 (two-audience summary), Section 11.4.5 (captured-evidence table), Section 11.4.6 (no-guessing), Section 11.4.44 (revision header), Section 11.4.65 (universal export).